



Building REST APIs in Python

Summary

- What are REST APIs?
- Designing REST APIs using HTTP standards
- Handling requests and responses
- Introduction to Flask
- Testing APIs with Postman
- Introduction to FastAPI

RESTful APIs

REST - REpresentational State Transfer

- Is an architectural model for distributed systems which consists of a set of constraints (recommendations) proposed to be used in developing Web Services.
- Roy Fielding described 6 principles that a system should adhere to be considered RESTful.

[Roy Fielding Thesis](#)

REST Principles

1. Uniform identification of resources

Every resource must be identifiable via a uniform resource identifier (URI)

Examples:

- `/users/123` identifies a specific user with ID 123.
- `/users` identifies the *collection* of all users.

URIs allows clients to locate and interact with resources without knowing the internal structure of the server, thus decoupling the client and server implementations.

REST Principles

2. Representation and manipulation of resources

The server never sends the resource itself, but a representation of it (JSON, XML, HTML).

Examples:

```
{ "id": 123, "name": "Alice" }
```

PUT /users/123

Content-Type: application/json

```
{ "name": "Alice Smith" }
```

REST Principles

3. Hypermedia As The Engine Of Application State (HATEOAS)

Communication between client and server is made between actions known to both sides, the client interacting with the server through hypermedia links provided by the representation they receive from the server, thus the client should not need additional information about the API – it should discover what it can do next from the resource's representation.

Example:

```
{ "id": 123,  
  "name": "Alice",  
  "_links": {  
    "self": { "type": "GET", "href": "/users/123" },  
    "update": { "type": "PATCH", "href": "/users/123" },  
    "delete_user": { "type": "DELETE", "href": "/users/123" },  
    "friends": { "type": "GET", "href": "/users/123/friends" }  
  }...  
}
```

REST Principles

4. Stateless Interactions

Every request from the client to the server must contain all the information necessary to understand and process it, needs to be processed independently and without reutilized data from other users.

Example:

GET /users/123/orders

Authorization: Bearer jwt_token

The token must be sent each time, since the server does not remember that the client logged in earlier.

REST Principles

5. Idempotent behavior

The processing of an action needs to be made in the same way no matter the number or order of requests made.

Examples:

GET – always return the same result without modifying the resource

PUT – sending the same update repeatedly results in the same resource state

DELETE – deleting something twice is the same as deleting it once, but the second time you might get a 404 Not found error

REST Principles

6. Intermediary entities

REST supports the use of intermediaries (proxies, caches, load balancers) to improve performance and reduce server load

The client does not need to know whether it's communicating directly with the server or through intermediaries.

Examples:

Caching on GET /products

- it's a public endpoint, requiring no additional information like authentication

Proxy load balancer distributing requests across multiple backend servers

REST Principles Now

1. Uniform Identification of Resources

Mostly followed, but it's often confused with RPC endpoints, and sometimes combined, because at the end of the day, the developer has full control over what he implements

Examples:

REST:

GET /users/123

POST /orders/{id}

RPC:

POST /getUser?id=123

POST /processPayment

Nice article that explains more on the differences:

[RPC vs REST - Difference Between API Architectures - AWS](#)

REST Principles Now

2. Representation and Manipulation of Resources

Today almost everything is JSON. It got so far that the header for content negotiation is often ignored (Accept: application/json) since most APIs consume JSON.

3. HATEOAS

Completely ignored. Developers prefer to read API docs and hardcode endpoints in code. Found unnecessary and hard to implement because of often usage of Swagger/OpenAPI.

It slows down the connection by offering bigger responses and often that additional information can be used against the application with malicious intent to gather additional information that is usually unwanted.

REST Principles Now

4. Stateless Interaction

True statelessness can be expensive if every request is to go through the authentication process. So today we use tokens based on cookies and session state, that expire after some time and before expiration is used in every request without going through the authentication process again.

5. Idempotent Behavior

HTTP behavior is mostly respected but

- PUT is often misused as a partial update (which should be PATCH).
- Some APIs use POST for everything, ignoring idempotence. (RPC)

6. Intermediary Entities

- All intermediary entities are often used these days, especially caching mechanism.
- But rare for dynamic REST responses like personalized data or states that change often.

Designing REST APIs with HTTP Standards

- Use **HTTP methods** meaningfully:
 - GET → Retrieve data
 - POST → Create data
 - PUT/PATCH → Update data
 - DELETE → Remove data
- **Status codes** matter:
 - 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error
- Use **resource-based URLs**:
 - /users, /products/123

Sending & Receiving Requests/Responses

- Clients (e.g., browsers or Postman) send HTTP requests
- Server returns a response:
 - Body: Data in JSON, XML, HTML, etc.
 - Headers: Metadata (Content-Type, Authorization)
 - Status Code

GET /api/users

Response: 200 OK

```
[  
  {"id": 1, "name": "Alice"},  
  {"id": 2, "name": "Bob"}  
]
```

Transferring Data Types

- Textual Data: JSON, XML, plain text
- Binary Data: Files (PDFs, images, etc.)
- Multimedia:
 - Use multipart/form-data for file uploads
 - Serve images using `send_file()` or `send_from_directory()` in Flask



Introduction to Flask

Introduction to Flask

- Flask is a micro web framework written in Python.
- It is designed to be lightweight, easy to use, and highly extensible.
- Perfect for building RESTful APIs, prototypes, and small to medium web services.

Flask

- Creating a Flask app

```
from flask import Flask  
app = Flask(__name__)
```

Flask

- common Flask environment variables for development

```
export FLASK_APP=app.py  
export FLASK_ENV=development  
export FLASK_DEBUG=1  
export FLASK_PORT=5000
```

Flask

- Defining routes

```
@app.route('/')  
def home():  
    return "Hello, Flask!"
```

Flask

- Running Flask app

```
if __name__ == '__main__':  
    app.run(debug=True)
```

- Accessing the app at <http://localhost:5000>

Flask

- Built on Jinja2 templates

```
from flask import Flask, render_template, request
```

```
@app.route('/')  
def index():  
    return render_template('index.html')
```

```
@app.route('/greet', methods=['POST'])  
def greet():  
    name = request.form['name']  
    return render_template('greet.html', name=name)
```

Flask

- Integration with SQLAlchemy

```
from flask_sqlalchemy import SQLAlchemy
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite://users.db'
```

```
db = SQLAlchemy(app)
```

Flask

- Integration with SQLAlchemy

```
from app import db

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80), nullable=False)

    def to_dict(self):
        return {"id": self.id, "name": self.name}

@app.route('/users', methods=['POST'])
def create_user():
    data = request.get_json()
    user = User(name=data['name'])
    db.session.add(user)
    db.session.commit()

    return jsonify(user.to_dict()), 201
```


Creating API Endpoints in Flask

- Define routes with methods:

```
@app.route('/api/users', methods=['GET'])  
def get_users():  
    return jsonify(users)
```

- Handle different request types (GET, POST, PUT, DELETE)
- Use request to access data sent by clients

GET a User

```
from flask import jsonify, request, abort

# Sample in-memory data
users = [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"}
]

@app.route('/api/users', methods=['GET'])
def get_users():
    return jsonify(users)
```

POST a User

```
@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.json
    new_user = {'id': len(users) + 1, 'name': data['name']}
    users.append(new_user)

    return jsonify(new_user), 201
```

DELETE a User

```
@app.route('/api/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    if user_id not in users:
        abort(404)
    del users[user_id]
    return '', 204
```

Testing with Postman

- Postman helps test your API without writing code
- Features:
 - Send GET/POST/PUT/DELETE requests
 - View responses, headers, status codes
 - Save collections and automate tests

Postman

HTTP <http://127.0.0.1:8000/check?database=true> [Save](#)

GET <http://127.0.0.1:8000/check?database=true> [Send](#)

Params • Authorization Headers (6) Body Pre-request Script Tests Settings [Cookies](#)

Query Params

	Key	Value	Bulk Edit
☒	database	true	
	Key	Value	

Body Cookies Headers (5) Test Results [200 OK](#) 8 ms 325 B [Save Response](#)

Pretty Raw Preview Visualize JSON [≡](#)

```
1 {
2   "status": true,
3   "application_info": {
4     "up_since": "2025-08-11T16:14:06.901226",
5     "name": "APP",
6     "environment": "local",
7     "build_version": "1.0.1",
8     "api_version": "1.0.0"
9   },
10  "third_parties": null
11 }
```



Introduction to FastAPI

What is FastAPI?

- Modern, fast, high-performance web framework for building APIs
- Auto-generates interactive API docs using OpenAPI standard (Swagger UI and ReDoc)
- Uses Pydantic models to validate and serialize/deserialize request data automatically
- Supports async and await allowing non-blocking code for better performance in I/O-heavy apps

FastAPI

- Creating a FastAPI app

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

FastAPI

- Defining routes

```
@app.get("/")  
def read_root():  
    return {"message": "Hello, FastAPI!"}
```

POST

- Usage of models to validate and serialize the request schema

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

users = []

@app.post("/users", status_code=201)
def create_user(user: User):
    users.append(user)
    return {"message": "User created", "user": user}
```

GET

- Declare model also for API response

```
from pydantic import BaseModel
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    description: str
```

```
    price: float
```

```
@app.get("/items/", response_model=list[Item])
```

```
def read_items():
```

```
    items = [Item(name="Foo", description="A new item", price=45.2),
```

```
              Item(name="Bar", description="Another item", price=10.5)]
```

```
    return items
```

FastAPI

- Powerful with async/await

```
import asyncio

@app.get("/wait")
async def wait_example():
    await asyncio.sleep(3)    # simulate a long-running task

    return {"message": "Thanks for waiting!"}
```

```
uvicorn main:app --reload
```

Flask vs FastAPI

Use Flask When:

- You're building a small API or prototype quickly
- Simplicity and flexibility are key
- You don't need async or auto docs
- You want a large ecosystem and community

Use FastAPI When:

- You need high performance or real-time features
- You prefer Python type hints and async support
- You want automatic input validation
- You need built-in API documentation (OpenAPI)

